

Class: MainMenu						
Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
loadGame	1	loadGame with a name with numbers in the string	Sam-Paul.txt	true	true	P
	2	loadGame with a name with numbers in the string	Hello1.txt	true		P
	3	loadGame where the player doesn't exists	Sam-Paul2.txt (the player doesn't exists)	false	false	P
blessCauldronPay	1	Has one unusable cauldron and the player has enough crystals	player's crystals 1000 and 1 unusable cauldron	true	true	P
	2	doesn't have any unusable cauldron and the player has enough crystals	player's crystals 1000 and 0 unusable cauldron	false	false	P
	3	The player doesn't have enough crystals	player's crystals 500 and 1 unusable cauldron	false	false	P
claimLoginBonus	1	Test when the player already claimed the bonus	loginBonusClaimed = true	false	false	P
	2	Test when the player haven't claimed the bonus yet and calls the method	loginBonusClaimed = false	true	true	P
	3	Exit the game and load the save again to test if the player could claim the bonus	loginBonusClaimed = false	true	true	P
parseSelectedSlots	1	Test when the inputted string has spaces in between the commas	"1 , 3 , 5 "	[1,3,5]	[1,3,5]	P
	2	Test when the inputted string when there is duplicated values	"2, 2, 4"	[2,4]	[2,4]	P
	3	Test when the inputted string has a number outside the range of 1-8	"6, 7, 9"	[6,7]	[6,7]	P
findAvailableListing	1	Test on whether the first listing listed would be found	slotNumber = 1	the listing at slot 1	the listing at slot 1	P
	2	Test on whether the last listing listed would be found	slotNumber = 8	the listing at slot 8	the listing at slot 8	P
	3	Test on a slot number that doesn't contain anymore listing	slotNumber =5 (but the player bought slot 1 prior without refreshing the market)	null	null	P
hasSelectedFruit	1	Test when the arrayList doesn't contain the key	fruits = ["Strawberry", "Orange", "Kiwi"] key = "Blueberry"	false	false	P
	2	Test when the arrayList contains the key at the start of the list	fruits = ["Orange", "Blueberry", "Banana"] key = "Orange"	true	true	P
	3	Test when arrayList is empty	fruits = [] key = "Strawberry"	false	false	P
	4	Test when the arrayList contains the key at the end of the list	fruits = ["Orange", "Blueberry", "Banana"] key = "Banana"	true	true	P
Class: Save						
Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
saveExists()	1	Existing default save Sam-Paul	Sam-Paul	true	true	P
	2	Different capitalization of default save	sAM-pAUL	true	true	P
	3	Incorrect save name	Paul-Sam	false	false	P
createDefaultPlayer	1	Creating new player	Default	Address of New Default Player	Address of New Default Player	P
loadPlayer	1	Existing default save Sam-Paul	Sam-Paul	Address of existing saved player	Address of existing saved player	P
	2	Different capitalization of default save	sAM-pAUL	Address of existing saved player	Address of existing saved player	P
	3	Non-existing save	Paul-Sam	Save file does not exist. null	Save file does not exist. null	P
savePlayer	1	Player saved with empty inventory and empty recipebook	currentPlayer	true	true	P
	2	Player saved with default inventory	currentPlayer	true	true	P
randItem	1	Generate a random ingredient not already owned.	Empty inventory.	Returns a random ItemStack	Returns a random ItemStack	P
	2	Try random item when all ingredients are already owned.	Inventory already contains every ingredient from ingredient	null	null	P
hasSave	1	Check if save folder has at least one .txt save file.	src/data/saves/ contains Sam-Paul.txt	true	true	P
	2	Check if no saves are available.	Empty src/data/saves/ folder.	false	false	P
Class: Ingredient						
Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
isEqual	1	Test with two ingredients that have the same name	this = "Kiwi" other = "Kiwi"	true	true	P
	2	Test with two ingredients that have different names	this = "Kiwi" other = "Orange"	false	false	P
	3	Test with two ingredients that have the same name but different with capitalization	this = "Strawberry" other = "STraWbErry"	true	true	P

getName	1	Gets ingredient name	Ingredient "STRAWBERRY"	"STRAWBERRY"	"STRAWBERRY"	P
isFruit	1	Check if ingredient is fruit	STRAWBERRY, true	true	true	P
isConcoctionBase	1	Check if ingredient is a base	SYRUP BASE, false	true	true	P
getBuyingPrice	1	Get buying price	STRAWBERRY, true, 50	50	50	P
getSellingPrice	1	Get selling price	STRAWBERRY, true, 50, 25	25	25	P
isEqual	1	same ingredient names with different casing	"STRAWBERRY" and "strawberry"	true	true	P
	2	different ingredients	"STRAWBERRY" and "ORANGE"	false	false	P
	3	Compare ingredient with null	"STRAWBERRY" and null	false	false	P
loadIngredients	1	Load all ingredients from CSV	src/data/ingredients.csv exists	ArrayList<Ingredient> with loaded ingredients	Ingredient list loaded	P
findIngredient	1	Find existing ingredient by name	"STRAWBERRY"	matching Ingredient object	STRAWBERRY object	P
	2	Search for missing ingredient	"DRAGONFRUIT"	null	null	P
	3	Search using null	null	null	null	P
Class: ItemStack	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
getIngredient	1	Get ingredient from stack	Strawberry Ingredient	STRAWBERRY	STRAWBERRY	P
getQuantity	1	Get current stack quantity	Quantity = 3	3	3	P
add	1	Add positive quantity	Current quantity 3, add 2	5	5	P
	2	adding zero quantity	Current quantity 3, add 0	3	3	P
	3	adding negative quantity	Current quantity 3, add -1	3	3	P
remove	1	Remove valid quantity	Current quantity 5, remove 2	true, 3	true, 3	P
	2	Remove more than available	Current quantity 3, remove 5	false, 3	false, 3	P
	3	Remove zero	Current quantity 3, remove 0	false, 3	false, 3	P
Class: Inventory	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
addItemStack	1	Add new ingredient stack	Add STRAWBERRY, quantity 3	STRAWBERRY = 3	STRAWBERRY = 3	P
	2	Add duplicate ingredient stack	Existing STRAWBERRY = 3, add 2	STRAWBERRY = 5	STRAWBERRY = 5	P
	3	Add invalid quantity	Add STRAWBERRY, quantity 0	no change	no change	P
removeIngredient	1	Remove valid quantity	STRAWBERRY = 5, remove 2	true	true	P
	2	Remove more than available	STRAWBERRY = 3, remove 5	false	false	P
getQuantity	1	Get owned ingredient quantity	STRAWBERRY = 3	3	3	P
	2	Get quantity of missing ingredient	ORANGE not in inventory	0	0	P
hasIngredients	1	Check if inventory has recipe ingredients	Inventory has SYRUP BASE = 1, STRAWBERRY = 1	true	true	P
	2	Check missing recipe ingredient	Inventory has base but no fruit	false	false	P
addCauldron	1	Add one cauldron	Total 3, usable 3	Total 4, usable 4	4, 4	P
damageCauldron	1	Damage one usable cauldron	Total 3, usable 3	true, 2	true, 2	P
	2	Damage when no usable cauldrons exist	Total 3, usable 0	false, 0 usable	false, 0 usable	P
blessCauldron	1	Restore one unusable cauldron	Total 3, usable 2	true, 3 usable	true, 3 usable	P
	2	blessing when all are usable	Total 3, usable 3	false	false	P
countTotalCauldrons	1	Count total cauldrons	Total 3	3	3	P
countUsableCauldrons	1	Count usable cauldrons	Usable 2	2	2	P

countUsableCauldrons	1	Count unusable cauldrons	Usable 2, total 3	1	1	P
getIngredientStack	1	Get copy of inventory list	Inventory with two stacks.	copy of stack list	stack list returned	P
setTotalCauldrons	1	Set total cauldron count	Set total to 5	total = 5	total = 5	P
setUsableCauldrons	1	Set usable cauldron count	Set usable to 4	usable = 4	usable = 4	P
Class: Recipe	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
getRequiredIngredients	1	Test if it gets the required ingredrints	N/A	ArrayList of the required ingredients	ArrayList of the required ingredients	P
getConcoctionId	1	Test if it gets the concoction ID	N/A	Id of the recipe	Id of the recipe	P
getBase	1	Test if it gets the base of the recipe	N/A	base of the recipe	base of the recipe	P
matches	1	Check same fruit but wrong bas	Base SYRUP BASE, fruits [STRAWBERRY]	true	true	P
	2	Check correct base but missing fruit	Base SYRUP BASE, fruits []	false	false	P
	3	Check fruit order does not matter	Recipe [STRAWBERRY, BLUEBERRY]; input [BLUEBERR	true	true	P
getFruits	1	Get required fruits	Recipe fruit list [STRAWBERRY]	copy of fruit list	copy of fruit list	P
getSaleValue	1	Get recipe sale value	Sale value 100	100	100	P
getRequiredQuantity	1	Check required base quantity	SYRUP BASE	1	1	P
	2	Check required fruit quantity	STRAWBERRY	1	1	P
	3	Check ingredient not in recipe	ORANGE	0	0	P
containsFruit	1	Check fruit included in recipe	STRAWBERRY	true	true	P
	2	Check fruit not included	ORANGE	false	false	P
Class: Listing						
Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
purchase	1	Buy ingredient listing with enough crystal	Player has 5000; listing costs 150	true	true	P
	2	Buy ingredient listing without enough crystals	Player has 10; listing costs 150	false	false	P
	3	Buy cauldron listing with enough crystal	Player has 5000; cauldron costs 3000	true	true	P
	4	Try buying already sold listing	Listing already purchased	false	false	P
markSold	1	Mark listing unavailable	Available listing	isAvailable = false	FALSE	P
isAvailable	1	Check listing availability	New listing	true	true	P
isIngredientListing	1	Check ingredient listing type	Strawberry listing	true	true	P
	2	Check cauldron listing type	Cauldron listing	false	false	P
isCauldronListing	1	Check cauldron listing type	Cauldron listing	true	true	P
getSlotNumber	1	Get listing slot	Slot 1	1	1	P
getIngredient	1	Get listed ingredient	Strawberry object listing	STRAWBERRY	STRAWBERRY	P
getQuantity	1	Get listing quantity	Quantity = 3	3	3	P
getUnitPrice	1	Get listing unit price	Unit price = 50	50	50	P
Class: Market						
Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
generateListings	1	Test whether the generated listings is from 1-8	Ingredients in the file	the generated listings is 1-8	the generated listings is 1-8	P
	2	Test whether the generated listings randomly adds a cauldron in the listing	Ingredients in the file	cauldron was found in the listing	cauldron was found in the listing	P

	3	Test whether the generated ingredients has the quantity of 1-5	Ingredients in the file	the listing contains only quantities from 1-5	the listing contains only quantities from 1-5	P
refresh	1	Tests refreshing an empty market.	N/A	generates new listings with 1-8 slots	generates new listings with 1-8 slots	P
	2	Tests refreshing when the market is still full	N/A	generates new listings with 1-8 slots	generates new listings with 1-8 slots	P
	3	Test refresh multiple times	N/A	generates new listings with 1-8 slots	generates new listings with 1-8 slots	
buyMultiple	1	Buy only only from one slot	slotNumbers=[1]	slotNumber 1 would be purchased by the player	slotNumber 1 would be purchased by the player	P
	2	Buy from multiple slots	slotNumbers=[1,4,5]	slotNumber 1,4,5 would be purchased by the player	slotNumber 1,4,5 would be purchased by the player	P
	3	Buy from multiple slots but with an invalid slo	slotNumbers=[1,3,9,-1]	slotNumber 1,3 would be purchased by the player	slotNumber 1,3 would be purchased by the player	P
sellMultiple	1	Selling one type of item with the same quantity as what the player has	Inventory has Strawberry x 3 and the player is selling Strawberry x 3	remove the items from the inventory and add the crystals to the player	remove the items from the inventory and add the crystals to the player	P
	2	Selling multiple items with the same quantity as what the player has	Inventory has Kiwi x 1, Banana x 2, Blueberry x 1 and the player is selling Kiwi x 1, Banana x 2, Blueberry x 1	remove the items from the inventory and add the total crystals to the player	remove the items from the inventory and add the total crystals to the player	P
	3	Selling items that has a higher quantity than what the player has	Inventory has Strawberry x 3 and the player is selling Strawberry x 7	Nothing	Nothing	P
getAvailableListings	1	returns the availableListng	N/A	returns the available listings through an ArrayList	returns the available listings through an ArrayList	P
hasBeenGenerated	1	return generated this session	N/A	returns the generatedThisSession	return the generatedThisSession	P
Class: Player						
Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
addCrystals	1	Add positive crystals	Current 5000 add 500	5500 Crystals	5500	P
	2	Add negative crystals	Current 5000 add -100	5000 Crystals	5000	P
	3	Add zero	Current 5000 add 0	5000 Crystals	5000	P
spendCrystals	1	Spend affordable amount	Current 5000 spend 1000	true, 4000 Crystals	true, 4000	P
	2	Spend more than available	Current 500 spend 1000	false, 500 Crystals	false, 500	P
canAfford	1	Check affordable amount	Crystals 5000, amount 1000	true	true	P
	2	Check unaffordable amount	Crystals 500, amount 1000	false	false	P
	3	Check invalid amount	amount 0 or negative	false	false	P
getName	1	Get player name	Player named Sam-Paul	Sam-Paul	Sam-Paul	P
getCrystals	1	Get crystals	Player with 5000 crystals	5000	5000	P
getInventory	1	Get inventory	Any player	Inventory object	Inventory object	P
getRecipeBook	1	Get recipe book	Any player	RecipeBook object	RecipeBook object	P
Class: RecipeBook						
Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
loadRecipes	1	Test when the file doesn't exists	no file	false	false	P
	2	Test when tthe file exists and is in the directory	recipeCatalog.csv	true	true	P
	3	Test when the directory doesn' textists	no folder	false	false	P
loadUnlockedRecipelds	1	a copy of lds stored in the instance variable of the object	Arraylist of ids	this.unlockedRecipelds = new ArrayList<>(ids)	this.unlockedRecipelds = new ArrayList<>(ids)	P
findRecipeByld	1	Test with a recipe id at the start of the arrayList	1	returns the first recipe in the .csv file	returns the recipe object with ID 1	P
	2	Test with a recipe id at the end of the arrayList	130	returns the last recipe in the .csv file	returns the recipe obejct with ID 130	P
	3	Test with an id outside of the arrayList	6767	null	null	P
findRecipe	1	Test with a fruit that no recipe uses and a base that no recipe uses	fruits="Durian" base = "chickenSoup"	null	null	P
	2	Test with a fruit that a recipe uses and a base that no recipe uses	fruits="Strawberry" base = "chickenSoup"	null	null	P

	3	Test with a fruit and base that matches a recipe	fruits ="Strawberry" base= lotion base	returns the recipe with the matching fruit and lotion base	returns the recipe with the matching fruit and lotion base	P
unlockRecipe	1	Test with a recipe id that isn't unlocked	UnlockedId = 2 Id = 67	true	true	P
	2	Test with a recipe id that isn't unlocked yet but with multiple unlockedIDs already	UnlockedId = 1,2,3,4,5,6,7,8,9,10 Id = 130	true	true	P
	3	Test with a recipe that is already unlocked	UnlockedId = 2 Id = 2	false	false	P
isUnlocked	1	Test with a recipe id that is not unlocked by the player	id= 20 (user hasn't unlocked recipe id 20 yet)	false	false	P
	2	Test with a recipe id that is unlocked by the player	id= 2 (user has unlocked recipe id 2)	true	true	P
	3	Test with a recipe id that doesn't exists	id =-1	false	FALSE	P
getUnlockedRecipelds	1	Gets the the unlock recipe ids	N/A	returns the unlock recipe ids of the object	returns the unlock recipe ids of the object	P
Class: Brew Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
brewRecipe	1	Test player with enough ingredients and has unlocked the recipe	player's inventory = ["Bubble Base", "Strawberry"] unlockedRecipe = 2 recipe id = 2	true	true	P
	2	Test player with not enough ingredients and but unlocked the recipe	player's inventory = [] unlockedRecipe = 2 recipe id = 2	false	false	P
	3	Test player with enough ingredients and but hasn't unlocked the recipe	player's inventory = ["Bubble Base", "Orange"] unlockedRecipes = 2 recipe id = 7	false	false	P
brewCreative	1	Test with a list of fruits that have a duplicate	fruits = ["Banana", "Mango", "Strawberry", "Mango"] Base = lotion base	false	false	P
	2	Test when the recipe doesn't exists	fruits = ["Orange", "Blueberry", "Kiwi"] = Perfume base	false	false	P
	3	Test when the player doesn't have enough ingredients	fruits = ["Strawberry"] Base = Perfume base = Strawberry x 0	false	false	P
	4	Test when the recipe exists and the player has picked the correct corresponding ingredients	fruits = ["Strawberry"] Base = Perfume base playerInv = Strawberry x 1, Perfume base x 1	true	true	P
validateUniqueFruits	1	Test with list of fruits that contain a duplicate fruit	fruits = ["Banana", "Mango", "Strawberry", "Mango"]	false	false	P
	2	Test with list of fruits that doesn't contain a duplicate fruit	fruits = ["Banana", "Strawberry", "Mango"]	true	true	P
	3	Test with list that contain multiple duplicates of fruit	fruits = ["Orange", "Kiwi", "Kiwi", "Mango"]	false	false	P
hasEnoughIngredients	1	Test when the players inventory doesn't have the fruit item corresponding to the recipe	recipe = Perfume base , Strawberry playersInv= Perfume base x 1	false	false	P
	2	Test when the players inventory doesn't have the base item	recipe = Perfume base , Strawberry playersInv= Strawberry x 1	false	false	P
	3	Test when the player has the necessary fruit and base item	recipe = Perfume base , Strawberry playersInv= Perfume base x1, Strawberry x 1	true	true	P
consumeIngredients	1	remove players ingredients with a recipe with 1 base and 1 fruit	lotion base, Strawberry	removes lotion base and strawberry from player's inventory	removes lotion base and strawberry from player's inventory	P
	2	remove players ingredients with a recipe with 1 base and 2 fruit	perfume base, Kiwi, Orange	removes perfume base, kiwi, and orange from the player's inventory	removes perfume base, kiwi, and orange from the player's inventory	P
	3	remove players ingredients with a recipe with 1 base and 3 fruit	bubble base, Kiwi, blueberry, Orange	removes bubble base, kiwi, blueberry and orange from the player's inventory	removes bubble base, kiwi, blueberry and orange from the player's inventory	P
Class: Controller Method	#	Test Description	Sample Input Data	Expected Output	Actual Output	P/F
hasAnySave	1	No save is in the directory	empty directory	FALSE	FALSE	P
	2	has multiple saves in the directory	a directory containing multiple saves	TRUE	TRUE	P
	3	has one save in the directory	a directory containing only one save .txt file	TRUE	TRUE	P
saveExists	1	check when the player's name has a save file	Sam-Paul	TRUE	TRUE	P
	2	check when the player's name doesn't have a save file	67676767	FALSE	FALSE	P
	3	check when the directory containing the save files is empty	empty directory	FALSE	FALSE	P
saveGame	1	there is a current player	current player exists	TRUE	TRUE	P

	2	there is no current player or no player has been loaded yet	no current player exists	FALSE	FALSE	P
getCurrentPlayer	1	test when there is a current player	N/A	currentPlayer	currentPlayer	P
getMarket	1	test whether it gets the Market attribute	N/A	market	market	P
getBrew	1	test whether it gets the Brew attribute	N/A	brew	brew	P
isLoginBonusClaimed	1	test when login bonus has been claimed	N/A	TRUE	TRUE	P
	2	test when login bonus hasn't been claimed	N/A	FALSE	FALSE	P
setLoginBonusClaimed	1	test whether it sets the login bonus	loginBonusClaimed = true	the attribute loginBonusClaimed becomes true	the attribute loginBonusClaimed becomes true	P
getBrewsSinceMarketVisit	1	Test whether it gets the current brewsSinceMarketVisit	N/A	returns brewsSinceMarketVisit	returns brewsSinceMarketVisit	P
resetMarketBrewCounter	1	Test whether it makes brewsSinceMarketVisit to reset	N/A	brewsSinceMarketVisit = 0	brewsSinceMarketVisit = 0	P
brewRecipe	1	Test whether it would be a success when there is no fruit	Perfume base	FALSE	FALSE	P
	2	Test whether it would be a success when there is no concoction base	Strawberry, Banana	FALSE	FALSE	P
	3	Teset whether it would be a success when the recipe is valid and there is a fruit and a concoction base	bubble base and Strawberry	TRUE	TRUE	P
brewCreative	1	Test whether it would be a success when there is no fruit	Perfume base	FALSE	FALSE	P
	2	Test whether it would be a success when there is no concoction base	Strawberry, Banana	FALSE	FALSE	P
	3	Teset whether it would be a success when the recipe exists with the right corresponding fruit and concoction base	Mango, kiwi, coconut, and emerald beach lotion	TRUE	TRUE	P
generateMarket	1	Test when the market hasn't been generated	N/A	It generates the market alongside the corresponding listings	It generates the market alongside the corresponding listings	P
	2	Test when the getBrewsSinceMarketVisit is less than 3 and the market has already been generated	N/A	It does nothing	It does nothing	P
	3	Test when the getBrewsSinceMarketVisit is equal to 3 and the market has already been generated	N/A	refreshes the market and resets brew count	refreshes the market and resets brew count	P
getAvailableListing	1	Test whether it returns all the available listings	N/A	returns all the available listings	returns all the available listings	P
sellItems	1	Test whether it sells the item	N/A	It sells the item and adds the item value to the player's crystals	It sells the item and adds the item value to the player's crystals	P
calculateTotalCost	1	Test when the inputted slot numbers are empty	SlotNumber = []	0	0	P
	2	Test when the inputted slot number is null	SlotNumber = null	0	0	P
	3	Test when the inputted slot number contains valid values and is available to be bought	SlotNumber = [1,3]	player spends 700 (According to the cost of the two slots)	player spends 700 (According to the cost of the two slots)	P
calculateTotalSellValue	1	Test when the inputted itemStacks are empty	items = []	players earns 0	players earns 0	P
	2	Test when the inputted itemStacks contains one item with one quantity	items = [lemon x1]	player earns 25 crystals	player earns 25 crystals	P
	3	Test when the inputted itemStacks contains one item with more than one quantity	items = [lemon x2]	player earns 50 crystals	player earns 50 crystals	P
isSlotAvailable	1	when the slotNumber is outside the range of 1-8	slotNumber = 9	FALSE	FALSE	P
	2	when the slotNumber is inside the range of 1-8 but not available	slotNumber = 4	FALSE	FALSE	P
	3	when the slotNumber is inside the range of 1-8 and listing is available	slotNumber = 1	TRUE	TRUE	P
buyItems	1	Test when the player could afford all the items based on the selected slot	slotNumber = [1,3,5] and playersCrystal > totalcost	[1,1,1]	[1,1,1]	P
	2	Test when the player couldn't afford all the items based on the selected slot	slotNumber = [1,3,5] and playersCrystal < totalcost	[-3,-3,-3]	[-3,-3,-3]	P
	3	Test when teh player is null and tries to buy the selected slotNumbers	slotNumber = [1,3,5]	[-3,-3,-3]	[-3,-3,-3]	P
blessCauldronPay	1	test when the player has more than 1000 crystals and theres is an unusable cauldron	playersCrystal = 1500 and unusable cauldron = 1	1	1	P
	2	test when the player doesnt have 1000 crystals and theres is an unusable cauldron	playersCrystal = 700 and unusable cauldron = 1	0	0	P
	3	test when the player has 1000 crystals but every cauldron is usable	playersCrystal = 1500 and unusable cauldron = 0	0	0	P
claimLoginBonus	1	Test when the player just logged in and hasn't collected it yet	N/A	"MANGO" the name of the ingredient that was given to the player	"MANGO" the name of the ingredient that was given to the player	P
	2	Test when the player already collected the login bonus	N/A	"claimed"	"claimed"	P
	3	Test when the player logged in collected it then exited the game and try to collect the login bonus again	N/A	"BANANA" the name of the ingredient that was given to the player	"BANANA" the name of the ingredient that was given to the player	P